

Resolución de la Actividad 3: Clase Virtual 02

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

1. Identificación de componentes críticos (Parte A)

Componente	Qué almacena o resuelve	¿Qué pasa si falla?	¿Qué pasa si responde con datos desactualizados?	Criticidad
Configuración del tenant	Logo, nombre, tema, estado del tenant (activo/inactivo)	Branding perdido; usuarios ven interfaz genérica o tienen problemas de acceso	Usuarios ven logo/tema viejo o acceden cuando el tenant está desactivado	Media
Definición del proceso	Nodos, conexiones, reglas de negocio, versión	No se pueden iniciar ni avanzar instancias nuevas según las reglas definidas	Se ejecutan flujos con reglas obsoletas (ej. validaciones o aprobaciones incorrectas)	Alta
Estado actual de instancia	Nodo actual, status, datos del payload, versión usada	Procesos quedan "congelados"; imposibilidad de consultar o continuar el flujo	Race conditions graves: doble aprobación, avance sobre estado viejo, transiciones inválidas	Crítica
Tareas humanas	Pendientes, asignadas, completadas, assignee, due date	No se generan ni completan tareas; backlog invisible y SLAs rotos	Dos usuarios toman la misma tarea o se marca como completada una ya resuelta	Alta
Eventos de auditoría	Historial timestamped de ejecución (event sourcing)	Pérdida total de trazabilidad y capacidad de auditoría/compliance	Historial con huecos o eventos fuera de orden temporal (dificulta debugging y auditorías)	Alta
Métricas operativas	Tiempos de ejecución, volúmenes, tendencias, bottlenecks	Dashboards y reportes de performance quedan vacíos	Decisiones de capacidad o mejora continua basadas en datos desactualizados	Baja-Media

Respuestas a las preguntas de la Parte A

1. ¿Cuál de estos componentes debería priorizar consistencia?
Estado actual de instancia y Definición del proceso. Un estado inconsistente genera errores de negocio graves (doble aprobación, flujos saltados) que son muy costosos de reparar manualmente.

2. ¿Cuál debería priorizar disponibilidad?
Configuración del tenant y Métricas operativas. Pueden degradarse temporalmente sin detener el flujo principal de los tenants (alta de proveedores, aprobación de compras o reclamos).
3. ¿Cuál podría aceptar consistencia eventual?
Eventos de auditoría y Métricas operativas. Se pueden reconstruir desde la fuente de eventos y toleran un delay de segundos o minutos sin impacto crítico.
4. ¿Cuál no debería perder datos bajo ninguna circunstancia?
Estado actual de instancia, Eventos de auditoría y Definición del proceso versionada. Son la fuente de verdad del “qué pasó y dónde está el proceso”. Su pérdida implica procesos huérfanos e imposibilidad de auditoría.
5. ¿Cuál puede degradarse temporalmente sin afectar el proceso principal?
Métricas operativas y Configuración visual del tenant. El branding o los reportes agregados pueden estar desactualizados brevemente sin afectar la ejecución del BPM.

2. Análisis CAP por componente (Parte B)

Componente	Opción CAP sugerida	Justificación técnica
Configuración del tenant	AP (balance configurable)	Alta disponibilidad para branding y acceso multi-tenant es prioritaria. Cambios de estado (activar/desactivar) pueden requerir consistencia, pero la mayoría de lecturas toleran eventualidad con TTL corto o notificaciones.
Definición del proceso	CP (con versionado estricto)	La definición debe ser idéntica en todos los nodos para evitar que instancias usen reglas diferentes simultáneamente. El versionado permite transiciones suaves. En partición es preferible bloquear nuevas instancias que servir una definición inconsistente.
Estado actual de instancia	CP	Es el corazón del motor de workflow BPM. Una lectura stale puede producir estados inválidos muy difíciles (casi imposibles) de reconciliar después. En partición de red es mejor rechazar operaciones que arriesgar inconsistencia de estado.
Tareas humanas	CP (con optimistic locking)	Asignación y completado de tareas deben ser atómicos para evitar que dos usuarios tomen la misma tarea o que se complete dos veces. Optimistic locking (versión/ETag) permite alta concurrencia sin bloqueos pesados.
Eventos de auditoría	AP	Son append-only. Se pueden aceptar escrituras locales durante una partición y reconciliar/merge después usando timestamps y vector clocks. Eventual consistency es aceptable si se preserva el orden causal.

Componente	Opción CAP sugerida	Justificación técnica
Métricas operativas	AP	Son agregados derivados de los eventos. Pueden calcularse de forma asíncrona y toleran una fuerte eventualidad sin impacto en el flujo operativo principal.

Preguntas guía respondidas

- ¿Qué componente debería evitar lecturas inconsistentes?
Estado actual de instancia y Tareas humanas.
- ¿Dónde es preferible responder aunque el dato no sea el último?
Métricas operativas y Configuración visual del tenant.
- ¿Qué ocurre si durante una partición de red no se puede confirmar el estado actual?
Se prioriza Consistency (CP): mejor bloquear el avance de la instancia que generar un estado inválido.
- ¿Qué componente puede reconstruirse a partir de eventos?
Métricas operativas (y parcialmente el estado de instancia mediante event sourcing ligero).
- ¿Qué componente debería bloquearse antes que aceptar una decisión incorrecta?
Estado actual de instancia y Tareas humanas.

3. Modelos de consistencia (Parte C)

Componente	Modelo de consistencia sugerido	Motivo
Definición publicada de proceso	Strong consistency	Las nuevas instancias deben ver exactamente la última versión publicada. Los cambios deben propagarse de forma consistente antes de permitir nuevas ejecuciones.
Estado actual de instancia	Strong consistency	Cualquier actor (humano o sistema) debe ver el estado real antes de decidir una transición. Evita forks en el flujo y estados inválidos.
Visualización del historial de eventos	Eventual consistency + Causal	El historial puede mostrar eventos con pequeño delay. Se preserva el orden causal para que no aparezcan efectos antes que sus causas.
Vista del usuario que acaba de completar una tarea	Session consistency	El usuario que ejecutó la acción debe verla reflejada inmediatamente en su sesión (read-your-writes + monotonic reads). Otros usuarios pueden tolerar delay (eventual).
Métricas agregadas de tiempos de ejecución	Eventual consistency	Los agregados (promedios, conteos, tendencias) se actualizan de forma asíncrona a partir de eventos. No es crítico que estén 100% actualizados al segundo.

Componente	Modelo de consistencia sugerido	Motivo
Configuración visual del tenant	Eventual consistency	El logo, colores y tema pueden tardar algunos segundos en propagarse a todos los nodos sin impacto en la lógica de negocio ni en los flujos operativos.

4. Análisis de situaciones problemáticas (Parte D)

Escenario 1: Tarea humana completada dos veces

Un aprobador hace clic dos veces en “Aprobar” porque la pantalla tarda en responder. El sistema recibe dos solicitudes casi simultáneas.

Preguntas:

1. ¿Qué problema de consistencia aparece?
Race condition + posible violación de idempotencia. Se genera un estado inconsistente si ambas solicitudes se procesan.
2. ¿Debe aceptarse la segunda operación?
No. Debe detectarse como duplicada y ser ignorada (o fusionada).
3. ¿Qué componente debería detectar la duplicación?
El componente de Estado actual de instancia (o el manejador de tareas) mediante optimistic locking o verificación de estado + versión antes de aplicar el cambio.
4. ¿Qué debería registrarse en auditoría?
Ambos intentos: el primero como TASK_COMPLETED exitoso y el segundo como TASK_COMPLETED_DUPLICATE_IGNORED incluyendo actor, timestamp, IP y resultado de la operación.

Propiedad CAP en tensión: Principalmente Consistency.

Riesgo: Doble ejecución de efectos colaterales (notificaciones enviadas dos veces, stock actualizado dos veces, etc.) y estado de instancia corrupto.

Decisión recomendada: Hacer la operación idempotente + optimistic concurrency control.

Impacto en FlowOps: Protege especialmente procesos críticos como tenant_beta (aprobación de compras).

Escenario 2: Lectura desactualizada del estado

Un usuario consulta una instancia y ve que está “pendiente”, pero otro usuario la aprobó hace pocos segundos.

Preguntas:

1. ¿Es aceptable esta inconsistencia temporal?
Sí, es aceptable temporalmente (segundos) en la mayoría de los casos.
2. ¿Depende del tipo de proceso?
Sí. Es más tolerable en tenant_gamma (reclamos internos) que en tenant_beta (aprobación de compras con impacto financiero).
3. ¿Qué modelo de consistencia sería conveniente?
Session consistency para el usuario que interactúa activamente + Causal consistency para viewers generales.
4. ¿Qué mensaje podría mostrar la interfaz?
> “El estado puede estar actualizándose. Última sincronización: hace 6 segundos.
> Este dato puede no reflejar la versión más reciente. Haz clic en ‘Refrescar’ para actualizar.”

Escenario 3: Partición de red entre nodos

Una parte del sistema puede recibir eventos, pero no puede comunicarse con el nodo que mantiene el estado actual de la instancia.

Preguntas:

1. ¿Se debería seguir aceptando eventos?
Depende de la estrategia elegida: en modo AP sí (se encolan localmente); en modo CP se rechazan hasta resolver la partición.
2. ¿Se debería bloquear el avance del proceso?
Sí. Recomendamos bloquear la instancia afectada (circuit breaker + marca de estado "EN_RECONCILIACIÓN").
3. ¿Qué se prioriza: disponibilidad o consistencia?
Se prioriza Consistency sobre Availability para el estado de instancia. Es preferible degradar disponibilidad temporalmente que generar un estado inválido de muy difícil reparación posterior.
4. ¿Qué datos deberían reconciliarse luego?
Eventos encolados en la partición aislada + replay ordenado por timestamp/vector clock + detección y resolución de conflictos (ej. dos transiciones sobre el mismo nodo).

Escenario 4: Configuración de proceso modificada durante una ejecución

Un administrador cambia la definición del proceso mientras hay instancias en curso.

Preguntas:

1. ¿Las instancias existentes deben seguir con la versión anterior?
Sí. Deben continuar con la versión con la que fueron iniciadas (process_version pinned en la instancia).
2. ¿Las nuevas instancias deben usar la nueva versión?
Sí. Una vez publicada la nueva versión, las nuevas instancias la utilizan.
3. ¿Qué riesgo aparece si no hay control de versiones?
Instancias a mitad de ejecución encuentran nodos, edges o condiciones que ya no existen → excepciones en runtime, procesos abortados, datos huérfanos y pérdida de trazabilidad.
4. ¿Este componente requiere consistencia fuerte?
Sí, el componente Definición del proceso requiere strong consistency (o al menos causal fuerte) al momento de publicar una nueva versión, para que todos los nodos vean exactamente la misma definición al mismo tiempo.

5. Decisiones preliminares para el TPO (Parte E)

Decisión	Respuesta preliminar	Justificación
¿Qué datos no pueden perderse?	Estado actual de instancias activas + Eventos de auditoría completos + Definiciones de procesos versionadas (histórico de cambios)	Son la fuente de verdad del flujo operativo y del compliance. Su pérdida implica procesos huérfanos e imposibilidad de realizar auditorías regulatorias.
¿Qué datos pueden ser eventualmente consistentes?	Métricas operativas y dashboards + Configuración visual y de tema de tenants + Listados agregados de historial (para usuarios no activos)	Pueden reconstruirse completamente desde los eventos de auditoría. Un delay de segundos o minutos no afecta la operación crítica del BPM.

Decisión	Respuesta preliminar	Justificación
¿Qué operación debería ser idempotente?	Completar una tarea humana (POST /tasks/{task_id}/complete) + Iniciar una instancia (reintentos por timeout de red) + Transiciones internas de nodo	Los clientes (UI o integraciones externas) pueden reintentar operaciones por fallos de red o doble clic. La idempotencia + unique constraint previene duplicados y mantiene el estado consistente.
¿Qué componente debería priorizar disponibilidad?	Configuración del tenant + Métricas operativas + Visualización de historial de eventos (para usuarios no activos en la instancia)	Estos componentes no bloquean el flujo principal. Un tenant puede seguir operando aunque su logo tarde en cargar o los reportes tarden unos segundos en refrescar.
¿Qué componente debería priorizar consistencia?	Estado actual de instancia + Tareas humanas (asignación y completado) + Definición del proceso (al momento de publicar una nueva versión)	Un estado inconsistente genera errores de negocio graves (doble aprobación, flujos saltados, datos corruptos). Es preferible degradar disponibilidad temporalmente (mostrar “sistema ocupado, reintente en unos segundos”) que aceptar un estado inválido.
¿Qué eventos deberían registrarse siempre?	INSTANCE_STARTED, NODE_ENTERED, TASK_CREATED, TASK_COMPLETED, NODE_EXITED, INSTANCE_COMPLETED, INSTANCE_FAILED, DECISION_EVALUATED	Permiten la reconstrucción completa del estado (event sourcing ligero), el cálculo de SLAs, debugging detallado y auditoría regulatoria completa. Deben ser durables y ordenados causalmente.

6. Conclusión

El análisis CAP aplicado a FlowOps revela que un sistema BPM distribuido multi-tenant debe adoptar un enfoque híbrido. Los componentes centrales como el estado de las instancias y las tareas humanas requieren priorizar consistencia fuerte (CP) para garantizar la corrección del flujo de trabajo y evitar estados inválidos cuyo costo de reparación es muy alto (especialmente en procesos de compras y reclamos). Por otro lado, componentes auxiliares como métricas operativas y configuración visual pueden inclinarse hacia disponibilidad (AP) con consistencia eventual, ya que toleran datos levemente desactualizados sin comprometer la integridad del proceso.

La aplicación de modelos de consistencia como session consistency para el usuario que interactúa activamente y eventual/causal consistency para historiales y métricas permite un balance práctico y realista. Decisiones clave como la idempotencia de operaciones de completado de tareas, el versionado estricto de definiciones de procesos y el registro exhaustivo de eventos preparan el terreno para una implementación robusta en clases posteriores.

Este análisis conceptual asegura que FlowOps pueda escalar manteniendo la confiabilidad esperada en una plataforma SaaS multi-tenant de automatización de procesos operativos, sentando las bases para la elección de motores NoSQL (MongoDB para documentos de proceso e instancia, Redis o Cassandra para estado y eventos de alta escritura) en las siguientes etapas del TPO.